

Bagian 1: Impor Library

```
In [1]: # --- START OF FILE dashboard_classic_combined.py (Tidak perlu diubah dari versi

import streamlit as st
import yfinance as yf
import numpy as np
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
from sklearn.linear_model import LogisticRegression, LinearRegression # Impor Li
from sklearn.ensemble import RandomForestClassifier # Impor juga jika pakai RF R
import joblib
import matplotlib.pyplot as plt
import os

# Coba impor pandas_ta
try:
    import pandas_ta as ta
except ModuleNotFoundError:
    st.error("ERROR: Library pandas_ta tidak ditemukan. Mohon install: pip insta
    st.stop()
else:
    PANDAS_TA_AVAILABLE = True
```

Penjelasan:

Blok ini adalah langkah awal yang memuat semua "perkakas" atau library Python yang dibutuhkan untuk menjalankan dashboard, memproses data, memuat model, dan menampilkan hasil.

- **import streamlit as st:** Mengimpor pustaka Streamlit, yang merupakan framework utama untuk membangun dan menjalankan aplikasi web interaktif ini. Semua elemen antarmuka pengguna (UI) seperti teks, tombol, grafik, dll., akan dibuat menggunakan fungsi-fungsi dari st.
- **import yfinance as yf:** Mengimpor pustaka yfinance. Di dashboard ini, fungsinya adalah untuk mengunduh data harga historis terbaru dari Yahoo Finance berdasarkan ticker yang dipilih atau default, yang nantinya akan digunakan sebagai input untuk prediksi.
- **import numpy as np:** Mengimpor pustaka NumPy. Dibutuhkan untuk operasi numerik yang efisien, terutama saat bekerja dengan array data (misalnya, saat menangani output dari scaler atau model).
- **import pandas as pd:** Mengimpor pustaka Pandas. Sangat penting untuk bekerja dengan data dalam format tabel (DataFrame), seperti data harga historis yang diunduh, melakukan perhitungan fitur teknikal, dan manipulasi data lainnya.
- **from sklearn.preprocessing import MinMaxScaler:** Mengimpor kelas MinMaxScaler dari Scikit-learn. Di sini, fungsinya adalah untuk memuat objek scaler yang sudah dilatih (di-fit) pada data pelatihan, dan menggunakan metode .transform() untuk menskalakan data input baru sebelum dimasukkan ke model.

- **from sklearn.linear_model import LogisticRegression, LinearRegression:** Mengimpor kelas model dari Scikit-learn. Meskipun model ini akan dimuat dari file, impor kelasnya tetap diperlukan agar joblib dapat merekonstruksi objek model dengan benar saat memuat file .pkl.
- **from sklearn.ensemble import RandomForestClassifier:** Sama seperti di atas, impor kelas RandomForestClassifier diperlukan agar joblib bisa memuat model RF yang tersimpan.
- **from sklearn.metrics import (...):** Mengimpor beberapa fungsi metrik (meskipun tidak semua digunakan secara aktif untuk menampilkan hasil evaluasi di dashboard ini, beberapa mungkin diperlukan secara internal atau untuk logika tambahan di masa depan). Kegunaan utamanya di skrip pelatihan, tetapi diimpor juga di sini mungkin untuk konsistensi atau penggunaan potensial.
- **import joblib:** Mengimpor pustaka Joblib. Fungsinya di dashboard ini adalah memuat (joblib.load) objek model dan scaler yang sudah dilatih dan disimpan sebelumnya dalam format file .pkl.
- **import matplotlib.pyplot as plt:** Mengimpor modul pyplot dari Matplotlib. Digunakan untuk membuat grafik harga historis dan menampilkan titik prediksi di dalam dashboard.
- **import os:** Mengimpor modul OS. Fungsinya di sini adalah untuk memeriksa keberadaan file model dan scaler (os.path.exists) sebelum mencoba memuatnya, untuk mencegah error.
- **import math:** Mengimpor modul math standar Python. Mungkin tidak banyak digunakan secara langsung dalam logika inti dashboard ini, tetapi bisa berguna untuk perhitungan matematika tambahan jika diperlukan.
- **try...except...else untuk pandas_ta:** Blok ini menangani dependensi opsional pandas_ta. Pustaka ini sangat membantu dalam menghitung berbagai indikator teknikal. Jika pandas_ta terinstal, PANDAS_TA_AVAILABLE akan True dan fitur TA lanjutan akan dihitung. Jika tidak terinstal, aplikasi akan menampilkan pesan error menggunakan st.error() dan berhenti (st.stop()) karena fitur input penting akan hilang, membuat prediksi tidak valid.

Bagian 2: Konfigurasi Dasar dan Nama File

```
In [2]: # --- Konfigurasi Dasar ---
DEFAULT_TICKER = 'BTC-USD'
TIME_STEP_CONTEXT = 60
BUFFER_DAYS = 55 # Sesuaikan buffer jika perlu

# --- Nama File Model & Scaler ---
LOGREG_MODEL_SAVE_PATH = f'logreg_clf_{DEFAULT_TICKER}_advTA_model.pkl'
RF_MODEL_SAVE_PATH = f'rf_clf_{DEFAULT_TICKER}_advTA_model.pkl'
# --- BARU: Path untuk model regresi dan scaler Y ---
LINREG_MODEL_SAVE_PATH = f'linreg_reg_{DEFAULT_TICKER}_advTA_model.pkl' # Asumsi
```

```

SCALER_Y_REG_SAVE_PATH = f'scaler_y_classic_reg_{DEFAULT_TICKER}_advTA.pkl'
# -----
SCALER_X_SAVE_PATH = f'scaler_x_classic_combined_{DEFAULT_TICKER}_advTA.pkl' # S

# --- Fitur Input (DARI NOTEBOOK TRAINING - HARUS SAMA PERSIS, SEKARANG 16) ---
FEATURE_COLUMNS_INPUT = ['BBB_20_2.0', 'CMF_20', 'Close', 'Close_Diff_lag1',
                          'Close_Diff_lag2', 'Close_Diff_lag3', 'MACD_12_26_9',
                          'OBV', 'RSI_14', 'SMA_20', 'STOCHd_14_3_3', 'STOCHk_14_
                          'Volume', 'Volume_lag1', 'Volume_lag2', 'Volume_lag3']
N_FEATURES_INPUT = len(FEATURE_COLUMNS_INPUT) # Akan jadi 16

```

Penjelasan Detail:

Bagian ini mendefinisikan parameter-parameter statis dan konstanta yang akan digunakan di seluruh skrip dashboard.

- **DEFAULT_TICKER** = 'BTC-USD': Menetapkan simbol aset default yang akan dianalisis dan diprediksi. Penting dicatat bahwa model dan scaler yang dimuat dari file (.pkl) juga harus sesuai dengan ticker ini (karena nama file mengandung DEFAULT_TICKER).
- **TIME_STEP_CONTEXT** = 60: Variabel ini mungkin tidak digunakan secara aktif dalam logika model klasik ini, seringkali relevan untuk model sekuensial (seperti LSTM) yang melihat n langkah waktu sebelumnya.
- **BUFFER_DAYS** = 55: Parameter penting untuk pengambilan data. Saat meminta data terbaru dari yfinance, skrip akan meminta data tambahan sebanyak BUFFER_DAYS sebelum tanggal yang dibutuhkan. Ini untuk memastikan ada cukup data historis untuk menghitung fitur berbasis window (misalnya SMA 20 hari butuh 20 data sebelumnya) dan fitur lag (misalnya lag 3 hari butuh 3 data sebelumnya) tanpa menghasilkan nilai NaN pada data paling akhir yang akan digunakan untuk prediksi.
- **Variabel _SAVE_PATH**: Mendefinisikan path lengkap ke file-file .pkl yang berisi objek model dan scaler yang telah disimpan oleh skrip pelatihan. F-string (f'...') digunakan untuk membuat nama file dinamis berdasarkan DEFAULT_TICKER. Skrip ini akan menggunakan path ini untuk memuat objek-objek tersebut.
- **FEATURE_COLUMNS_INPUT**: Ini adalah daftar (list) Python yang berisi nama-nama string dari persis 16 fitur yang dijadikan input saat model dilatih. Penting: Daftar ini harus benar-benar identik dengan yang digunakan di notebook pelatihan (OP.ipynb). Baik nama fitur maupun jumlahnya (16) harus sama agar data yang diproses di dashboard cocok dengan input yang diharapkan oleh scaler_X dan model-model yang dimuat.
- **N_FEATURES_INPUT**: Menghitung jumlah item dalam FEATURE_COLUMNS_INPUT (seharusnya 16) dan menyimpannya untuk validasi nanti, memastikan konsistensi.

Bagian 3: Pengaturan Halaman Aplikasi & Judul

```
In [3]: st.set_page_config(page_title=f"Prediksi {DEFAULT_TICKER} (Arah & Harga)", layout
st.title(f" Dashboard Prediksi Harga {DEFAULT_TICKER} (Model Klasik - Arah & I
st.markdown("Dashboard ini menggunakan model Klasik untuk memprediksi **Arah** (
```

```
2025-04-28 14:58:16.738 WARNING streamlit.runtime.scriptrunner_utils.script_run_c
ontext: Thread 'MainThread': missing ScriptRunContext! This warning can be ignore
d when running in bare mode.
```

```
2025-04-28 14:58:16.748 WARNING streamlit.runtime.scriptrunner_utils.script_run_c
ontext: Thread 'MainThread': missing ScriptRunContext! This warning can be ignore
d when running in bare mode.
```

```
2025-04-28 14:58:18.041
```

Warning: to view this Streamlit app on a browser, run it with the following command:

```
streamlit run C:\Users\arest\AppData\Roaming\Python\Python311\site-packages\i
pykernel_launcher.py [ARGUMENTS]
```

```
2025-04-28 14:58:18.044 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
```

```
2025-04-28 14:58:18.046 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
```

```
2025-04-28 14:58:18.048 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
```

```
Out[3]: DeltaGenerator()
```

Penjelasan Detail:

Bagian ini menginisialisasi tampilan dasar aplikasi web menggunakan fungsi-fungsi dari Streamlit (st).

- **st.set_page_config(...):** Fungsi ini dipanggil sekali di awal untuk mengkonfigurasi metadata dan layout halaman.
 - **page_title:** Mengatur teks yang akan muncul di tab browser.
 - **layout="wide":** Menginstruksikan Streamlit untuk menggunakan lebar penuh browser untuk menampilkan konten, bukan layout terpusat yang lebih sempit. Ini berguna untuk dashboard yang menampilkan data atau grafik lebar.
- **st.title(...):** Menampilkan teks sebagai judul utama (heading level 1) di bagian atas aplikasi. Emoji  ditambahkan untuk visual.
- **st.markdown(...):** Menampilkan blok teks. Fungsi ini mendukung sintaks Markdown, memungkinkan format seperti bold (menggunakan ...), italic, list, dll., untuk memberikan deskripsi singkat tentang tujuan dashboard kepada pengguna.

Bagian 4: Fungsi Pemuatan Model dan Scaler (Menggunakan Cache)

```
In [4]: # --- Fungsi Helper ---

@st.cache_resource
def load_all_models_and_scalers(ticker): # Memuat SEMUA model & scaler
    """Memuat model Klasifikasi, Regresi, dan Scaler X & Y."""
    logreg_path = f'logreg_clf_{ticker}_advTA_model.pkl'
    rf_path = f'rf_clf_{ticker}_advTA_model.pkl'
    linreg_path = f'linreg_reg_{ticker}_advTA_model.pkl' # Path model regresi
```

```

scaler_x_path = f'scaler_X_classic_combined_{ticker}_advTA.pkl' # Sesuaikan
scaler_y_path = f'scaler_y_classic_reg_{ticker}_advTA.pkl' # Path scaler Y r

st.write("--- Memulai Pemuatan Model & Scaler ---")
files_ok = True
if not os.path.exists(logreg_path): st.error(f"File model LR tidak ditemukan")
if not os.path.exists(rf_path): st.error(f"File model RF tidak ditemukan: {r")
if not os.path.exists(linreg_path): st.error(f"File model Regresi tidak dite")
if not os.path.exists(scaler_x_path): st.error(f"File scaler X tidak ditemuk")
if not os.path.exists(scaler_y_path): st.error(f"File scaler Y (Regresi) tid")

if not files_ok:
    st.warning("Pastikan SEMUA file model (LogReg, RF, LinReg) dan scaler (X
    st.stop()

st.write("✓ Semua file model dan scaler ditemukan.")

try:
    model_logreg_loaded = joblib.load(logreg_path); st.write("✓ Model Logi:
    model_rf_loaded = joblib.load(rf_path); st.write("✓ Model Random Forest
    model_linreg_loaded = joblib.load(linreg_path); st.write("✓ Model Linea
    scaler_X_loaded = joblib.load(scaler_x_path); st.write("✓ Scaler X (Inp
    scaler_y_reg_loaded = joblib.load(scaler_y_path); st.write("✓ Scaler Y

    n_features_scaler = getattr(scaler_X_loaded, 'n_features_in_', None)
    # Validasi jumlah fitur yang diharapkan (16)
    if n_features_scaler is not None and n_features_scaler != N_FEATURES_INP
        st.error(f"Scaler X tidak valid (fitur: {n_features_scaler}, dihar")
    if n_features_scaler is not None: st.write(f"✓ Scaler X dikonfirmasi m")

    st.write("--- Selesai Pemuatan ---")
    return model_logreg_loaded, model_rf_loaded, model_linreg_loaded, scaler
except Exception as e: st.error(f"Gagal memuat model atau scaler: {e}"); st.

```

Penjelasan Detail:

Fungsi `load_all_models_and_scalers` ini bertugas memuat artefak machine learning yang sudah jadi (disimpan dari notebook pelatihan) ke dalam memori aplikasi.

- **@st.cache_resource:** Decorator Streamlit ini adalah kunci optimasi. Saat aplikasi dijalankan, fungsi ini akan dieksekusi. Hasilnya (objek model dan scaler yang dimuat) akan disimpan dalam cache khusus. Jika pengguna berinteraksi dengan UI (misalnya menekan tombol) yang memicu rerun skrip, Streamlit akan memeriksa cache. Jika input ke fungsi (ticker, dalam hal ini selalu `DEFAULT_TICKER`) dan kode fungsi tidak berubah, Streamlit akan langsung mengembalikan objek dari cache tanpa perlu menjalankan ulang logika pemuatan file dari disk. Ini secara drastis mempercepat respons aplikasi setelah pemuatan awal. `cache_resource` cocok untuk objek yang tidak mudah di-serialisasi seperti model ML.
- **Pengecekan File (`os.path.exists`):** Sebelum `joblib.load`, skrip secara defensif memeriksa apakah setiap file `.pkl` yang diharapkan ada. Ini memberikan pesan error (`st.error`) yang lebih informatif kepada pengguna/pengembang jika ada file yang hilang, dan menghentikan aplikasi (`st.stop()`) daripada crash karena file tidak ditemukan.

- **Pemuatan (joblib.load):** Fungsi inti dari joblib yang membaca file biner .pkl dan merekonstruksi objek Python asli (model LogisticRegression, RandomForestClassifier, LinearRegression, dan MinMaxScaler) di memori.
- **Validasi Scaler X (n_features_in_):** Setelah memuat scaler_X, kode ini mencoba mengakses atribut n_features_in_. Atribut ini secara otomatis ditambahkan oleh Scikit-learn saat scaler di-fit dan menyimpan jumlah fitur yang dilihatnya. Kode ini membandingkan nilai ini dengan N_FEATURES_INPUT (yang seharusnya 16). Jika tidak cocok, ini menandakan kemungkinan besar ada ketidaksesuaian antara fitur yang digunakan saat melatih scaler/model dan fitur yang didefinisikan dalam FEATURE_COLUMNS_INPUT di dashboard ini. Ini adalah sanity check penting untuk mencegah error saat .transform() dipanggil nanti.
- **Logging (st.write):** Pesan-pesan ini dicetak ke antarmuka Streamlit selama proses pemuatan, memberikan umpan balik visual tentang langkah mana yang sedang berjalan dan apakah berhasil.
- **Return:** Fungsi mengembalikan semua objek yang telah dimuat dan divalidasi, siap digunakan oleh bagian lain dari aplikasi.

Bagian 5: Fungsi Pengambilan dan Pemrosesan Data Terbaru

```
In [5]: # --- Fungsi get_and_process_data (Versi v3 yang sudah diperbaiki) ---
def get_and_process_data(ticker='BTC-USD', num_days_needed=100):
    # ... (Fungsi ini seharusnya sudah benar dari revisi sebelumnya) ...
    st.write(f"--- Memulai get_and_process_data (ticker: {ticker}, days needed:
start_date = (pd.Timestamp.today() - pd.Timedelta(days=num_days_needed * 1.5
end_date = (pd.Timestamp.today() + pd.Timedelta(days=1)).strftime('%Y-%m-%d'
    try: df = yf.download(ticker, start=start_date, end=end_date, progress=False
    except Exception as e: st.warning(f"Gagal ambil data yfinance: {e}"); return
    if df.empty: st.warning(f"Data yfinance kosong untuk {ticker}."); return None
    st.write(f"Data mentah diambil, shape: {df.shape}")
    df_processed = df.copy()
    if isinstance(df_processed.columns, pd.MultiIndex):
        st.write("MultiIndex kolom. Meratakan...");
        try:
            df_processed.columns = df_processed.columns.get_level_values(0).str.
            df_processed = df_processed.loc[:, ~df_processed.columns.duplicated(
            st.write(f"Kolom setelah perataan: {df_processed.columns.tolist()}")
        except Exception as e: st.error(f"Gagal meratakan MultiIndex: {e}."); re
    else:
        st.write("Menstandarkan nama kolom ke Title Case...")
        df_processed.columns = [str(col).title() for col in df_processed.columns
        st.write(f"Kolom setelah standarisasi: {df_processed.columns.tolist()}")
        high_col_name, low_col_name, close_col_name, volume_col_name = 'High', 'Low'
        required_cols = [high_col_name, low_col_name, close_col_name, volume_col_nam
        missing_req = [col for col in required_cols if col not in df_processed.colum
        if missing_req: st.error(f"Kolom standar hilang: {missing_req}."); return No
        st.write(f"Kolom HLCV standar digunakan: {required_cols}")
        st.write("Menghitung fitur teknikal..."); sma_col_name, rsi_col_name, macd_c
        try: df_processed[sma_col_name] = df_processed[close_col_name].rolling(windo
        except Exception as e: st.warning(f"Error SMA: {e}")
        if PANDAS_TA_AVAILABLE:
            try: df_processed.ta.rsi(close=df_processed[close_col_name], length=14,
            except Exception as e: st.warning(f"Gagal RSI: {e}")
```

```

    try: df_processed.ta.macd(close=df_processed[close_col_name], col_names=
except Exception as e: st.warning(f"Gagal MACD: {e}")
st.write("Menghitung fitur TA lanjutan...")
adv_ta_to_calculate = {'atr': {'high': df_processed[high_col_name], 'low
for func_name, params in adv_ta_to_calculate.items():
    try: getattr(df_processed.ta, func_name)(**params, append=True)
    except Exception as ta_err: st.warning(f" Gagal hitung TA ({func_nam
st.write("Fitur TA lanjutan selesai.")
else: st.warning("pandas_ta tidak tersedia.")
st.write("Menghitung fitur lag...")
target_diff_temp_col = 'Close_Diff_Temp'
df_processed[target_diff_temp_col] = df_processed[close_col_name].diff()
lags_to_add = [1, 2, 3]
for lag in lags_to_add:
    lag_col_diff = f'Close_Diff_lag{lag}'; df_processed[lag_col_diff] = df_p
    lag_col_vol = f'{volume_col_name}_lag{lag}'; df_processed[lag_col_vol] =
df_processed = df_processed.drop(columns=[target_diff_temp_col], errors='ign
st.write(f"Fitur lag dihitung.")
# --- PERBAIKAN: Gunakan FEATURE_COLUMNS_INPUT untuk final_cols_needed ---
final_cols_needed = FEATURE_COLUMNS_INPUT[:]
if 'Close' not in final_cols_needed: final_cols_needed.append('Close')
available_cols = [col for col in final_cols_needed if col in df_processed.co
missing_input_cols = [col for col in FEATURE_COLUMNS_INPUT if col not in ava
if missing_input_cols:
    st.warning(f"Fitur input hilang: {missing_input_cols}.")
    # Tetapkan fitur aktual yang akan digunakan berdasarkan yang tersedia
    st.session_state.feature_columns_input_actual = [col for col in FEATURE
    if len(st.session_state.feature_columns_input_actual) != N_FEATURES_INP
        st.error(f"Jumlah fitur input ({len(st.session_state.feature_colum
        return None
else:
    # Jika semua fitur ada, gunakan list asli
    st.session_state.feature_columns_input_actual = FEATURE_COLUMNS_INPUT[:

df_final = df_processed[available_cols].dropna()
st.write(f"Kolom akhir dipilih ({len(st.session_state.feature_columns_input_
if len(df_final) < 1: st.warning(f"Data tidak tersisa untuk {ticker}."); ret
st.write(f"Shape data akhir siap pakai: {df_final.shape}")
st.write("--- Selesai get_and_process_data ---")
return df_final
# --- AKHIR FUNGSI ---

```

Penjelasan Detail:

Fungsi `get_and_process_data` ini melakukan langkah-langkah krusial untuk menyiapkan data input terbaru agar sesuai dengan format yang diharapkan oleh model yang telah dilatih.

- **Pengambilan Data (yf.download):** Mengunduh data OHLCV dari Yahoo Finance untuk ticker yang diberikan. `start_date` dihitung mundur cukup jauh (dengan `BUFFER_DAYS`) untuk memastikan data awal cukup untuk perhitungan fitur berbasis window/lag. `end_date` diatur ke hari berikutnya untuk memastikan data hari ini (jika pasar sudah tutup) atau hari kerja terakhir masuk. `progress=False` menonaktifkan progress bar download yfinance di terminal.
- **Pemrosesan Kolom:** Melakukan standarisasi nama kolom (meratakan MultiIndex jika perlu, mengubah ke Title Case) agar konsisten ('High', 'Low', 'Close', 'Volume').

- **Rekayasa Fitur (Identik dengan Training):** Bagian ini harus mereplikasi persis bagaimana fitur dihitung di notebook pelatihan (OP.ipynb). Ini termasuk:
 - Menghitung SMA 20.
 - Menghitung RSI(14), MACD(12,26,9), ATR(14), Bollinger Bands(20,2), Stochastic(14,3,3), OBV, CMF(20) menggunakan `pandas_ta` (jika `PANDAS_TA_AVAILABLE` adalah `True`). Penggunaan `getattr` memungkinkan pemanggilan fungsi `pandas_ta` secara dinamis dari dictionary `adv_ta_to_calculate`.
 - Menghitung `Close_Diff` (perbedaan harga harian).
 - Menghitung fitur lag 1, 2, dan 3 hari untuk `Close_Diff` dan Volume menggunakan `.shift()`.
- **Seleksi & Validasi Fitur Input Akhir:**
 - Kode ini secara cermat memeriksa kolom mana dari `FEATURE_COLUMNS_INPUT` yang benar-benar berhasil dibuat dalam DataFrame `df_processed`.
 - Jika ada fitur input yang hilang (misal karena error `pandas_ta`), daftar fitur yang benar-benar tersedia disimpan ke `st.session_state.feature_columns_input_actual`. `st.session_state` adalah dictionary khusus Streamlit yang menyimpan nilai variabel antar rerun skrip (misalnya, saat pengguna menekan tombol). Ini memastikan bahwa saat prediksi nanti, aplikasi menggunakan daftar fitur yang benar dan konsisten dengan data yang ada.
 - Dilakukan pengecekan terakhir untuk memastikan jumlah fitur aktual yang akan digunakan sama dengan `N_FEATURES_INPUT` (16). Jika tidak, aplikasi berhenti.
- **dropna():** Menghapus baris-baris di awal data yang memiliki nilai NaN (hasil dari perhitungan rolling window atau shift).
- **Return:** Mengembalikan DataFrame `df_final` yang kolom-kolomnya sesuai dengan fitur input yang valid ditambah kolom 'Close', dan tidak lagi mengandung NaN.

Bagian 6: Logika Utama Aplikasi - Pemuatan Awal dan Sidebar UI

Bagian 7: Logika Utama Aplikasi - Proses Prediksi dan Tampilan Hasil

Pada bagian 6 dan bagian 7 penjelasan akan sedikit rumit karena proses telah menjadi 1 antara ui streamlit dan logika prediksi, saya akan mencoba untuk membahas sesuai pemahaman logika saya. Apabila anda tidak dapat memahami secara sepenuhnya maka akan saya akan berikan sebuah pesudocode agar anda bisa memahami alur kerjanya

```
In [6]: # --- Main App Logic ---
        model_logreg, model_rf, model_linreg, scaler_X, scaler_y_reg = load_all_models_a
```

```

st.sidebar.header("Pengaturan Data")
ticker_symbol = st.sidebar.text_input("Simbol Ticker", DEFAULT_TICKER)
data_period_display = st.sidebar.selectbox(
    "Periode Grafik Historis", ["90d", "6mo", "1y", "2y"], index=2, key="periode"
)
st.sidebar.caption(f"Prediksi didasarkan pada data historis terakhir (membutuhkan)")

if st.sidebar.button("Muat Data & Prediksi", key="prediksi_button"):
    if ticker_symbol != DEFAULT_TICKER:
        st.warning(f"Model/Scaler diload untuk {DEFAULT_TICKER}. Cek kesesuaian")

    data_processed = get_and_process_data(ticker=ticker_symbol, num_days_needed=

    if data_processed is not None and not data_processed.empty:
        st.subheader(f"Data Historis Terbaru ({ticker_symbol})")
        display_limit = int(min(90, len(data_processed)))
        # Tampilkan fitur aktual yang digunakan
        st.dataframe(data_processed[st.session_state.feature_columns_input_aktua
        try:
            last_actual_close_price = float(data_processed['Close'].iloc[-1])
            st.metric(label="Harga Penutupan Terakhir (Aktual)", value=f"${last_
        except (IndexError, KeyError) as e: st.error(f"Gagal ambil harga 'Close'
        except Exception as e: st.error(f"Error ambil harga terakhir: {e}"); st.

        st.write("--- Memulai Proses Prediksi (Klasik - Arah & Harga) ---")
        try:
            # --- Siapkan Input (gunakan list fitur aktual dari session state) -
            X_latest_unscaled = data_processed[st.session_state.feature_columns_
            st.write(f"Fitur input terakhir (unscaled): shape {X_latest_unscaled
            if X_latest_unscaled.empty: st.error("Gagal mendapatkan baris fitur

            # Pastikan jumlah kolom X_latest_unscaled cocok dengan scaler
            if X_latest_unscaled.shape[1] != scaler_X.n_features_in_:
                st.error(f"Jumlah fitur data terbaru ({X_latest_unscaled.shape[
                st.stop()

            X_latest_scaled = scaler_X.transform(X_latest_unscaled)
            st.write("Fitur input terakhir discaling.")

            st.write("Melakukan prediksi arah...")
            pred_logreg = model_logreg.predict(X_latest_scaled)[0]
            proba_logreg = model_logreg.predict_proba(X_latest_scaled)[0]
            pred_rf = model_rf.predict(X_latest_scaled)[0]
            proba_rf = model_rf.predict_proba(X_latest_scaled)[0]
            direction_map = {0: "Turun / Sama", 1: "Naik"}
            pred_direction_logreg = direction_map.get(pred_logreg, "Error")
            pred_direction_rf = direction_map.get(pred_rf, "Error")

            st.write("Melakukan prediksi harga (estimasi perubahan)...")
            scaled_diff_pred = model_linreg.predict(X_latest_scaled)
            st.write("Prediksi Scaled Difference (Regresi):", scaled_diff_pred)
            unscaled_diff_pred = scaler_y_reg.inverse_transform(scaled_diff_pred)
            st.write("Predicted Difference (Unscaled):", unscaled_diff_pred)
            predicted_absolute_price = last_actual_close_price + unscaled_diff_p
            st.write("Predicted Price (Absolute):", predicted_absolute_price)

            st.subheader("🔮 Prediksi Harga Berikutnya")
            st.markdown("**Prediksi Arah:**")
            col1, col2 = st.columns(2)
            with col1:

```

```

        st.metric(label="Logistic Regression", value=pred_direction_log)
        st.progress(float(proba_logreg[1]))
        st.caption(f"Prob. Naik: {proba_logreg[1]:.2%}")
        st.caption(f"Prob. Turun/Sama: {proba_logreg[0]:.2%}")
    with col2:
        st.metric(label="Random Forest", value=pred_direction_rf)
        st.progress(float(proba_rf[1]))
        st.caption(f"Prob. Naik: {proba_rf[1]:.2%}")
        st.caption(f"Prob. Turun/Sama: {proba_rf[0]:.2%}")

    st.divider()

    st.markdown("**Estimasi Harga (Regresi):**")
    change = predicted_absolute_price - last_actual_close_price
    delta_perc = (change / last_actual_close_price) * 100 if last_actual
    st.metric(label="Prediksi Harga Berikutnya",
              value=f"${predicted_absolute_price:,.2f}",
              delta=f"{change:,.2f} ({delta_perc:.2f}%)")
    st.caption("Estimasi ini berasal dari model regresi terpisah (misal:

    st.subheader("📊 Grafik Harga Historis & Estimasi Harga")
    days_for_plot = 90 if '90d' in data_period_display else 180 if '6mo'
    data_plot_display = get_and_process_data(ticker=ticker_symbol, num_d

    if data_plot_display is not None and not data_plot_display.empty and
        fig, ax = plt.subplots(figsize=(12, 6))
        plot_data_to_show = data_plot_display.tail(days_for_plot)
        ax.plot(plot_data_to_show.index, plot_data_to_show['Close'], val
        last_date_processed = data_processed.index[-1]
        next_date = last_date_processed + pd.Timedelta(days=1)
        ax.plot(next_date, predicted_absolute_price, 'ro', markersize=8
        ax.axhline(predicted_absolute_price, color='red', linestyle='--
        ax.set_title(f"Grafik Harga {ticker_symbol} ({data_period_displ
        ax.set_xlabel("Tanggal"); ax.set_ylabel("Harga (USD)")
        ax.legend(); ax.grid(True); fig.autofmt_xdate()
        st.pyplot(fig)
    else: st.warning("Tidak dapat menampilkan grafik historis.")

    except Exception as e: st.error(f"Terjadi kesalahan saat prediksi:"); st
    else: st.warning(f"Tidak ada data yang dapat diproses untuk {ticker_symbol}.
    else: st.info("Masukkan simbol ticker dan klik 'Muat Data & Prediksi'.")

# --- END OF FILE ---

```

11/15

```
2025-04-28 15:34:43.213 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2025-04-28 15:34:43.216 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2025-04-28 15:34:43.218 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2025-04-28 15:34:43.219 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2025-04-28 15:34:43.220 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2025-04-28 15:34:43.222 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2025-04-28 15:34:43.223 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2025-04-28 15:34:43.225 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2025-04-28 15:34:43.227 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2025-04-28 15:34:43.230 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2025-04-28 15:34:43.233 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2025-04-28 15:34:43.236 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2025-04-28 15:34:43.237 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2025-04-28 15:34:43.240 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2025-04-28 15:34:43.243 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2025-04-28 15:34:43.245 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2025-04-28 15:34:43.248 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2025-04-28 15:34:43.251 Session state does not function when running a script wit
hout `streamlit run`
2025-04-28 15:34:43.256 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2025-04-28 15:34:43.258 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2025-04-28 15:34:43.260 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2025-04-28 15:34:43.262 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2025-04-28 15:34:43.267 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2025-04-28 15:34:43.269 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2025-04-28 15:34:43.273 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2025-04-28 15:34:43.276 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2025-04-28 15:34:43.281 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2025-04-28 15:34:43.284 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2025-04-28 15:34:43.290 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2025-04-28 15:34:43.294 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
```

```

2025-04-28 15:34:43.296 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2025-04-28 15:34:43.300 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2025-04-28 15:34:43.305 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2025-04-28 15:34:43.307 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2025-04-28 15:34:43.310 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.

```

Penjelasan Detail (Bagian 6: Logika Utama Aplikasi - Pemuatan Awal dan Sidebar UI):

Bagian ini menginisialisasi variabel model/scaler dan membangun antarmuka pengguna (UI) di sidebar.

- **Pemuatan Model/Scaler:** Baris `model_logreg, ... = load_all_models_and_scalers(DEFAULT_TICKER)` memanggil fungsi yang sudah dijelaskan (Bagian 4) untuk memuat semua objek model dan scaler yang diperlukan. Karena fungsi ini menggunakan `@st.cache_resource`, proses pemuatan dari file `.pkl` hanya terjadi saat aplikasi pertama kali dijalankan atau jika cache tidak valid, membuat aplikasi lebih responsif pada interaksi berikutnya. Objek yang dimuat (model LR, RF, LinReg, scaler X, scaler Y regresi) disimpan dalam variabel Python untuk digunakan nanti.
- **Elemen Sidebar (`st.sidebar.*`):** Streamlit menyediakan objek `st.sidebar` untuk menempatkan elemen UI di panel samping yang terpisah dari area konten utama.
 - **`st.sidebar.header(...)`:** Menampilkan teks "Pengaturan Data" sebagai judul di dalam sidebar.
 - **`st.sidebar.text_input(...)`:** Membuat sebuah kotak input teks di sidebar dengan label "Simbol Ticker" dan nilai defaultnya diisi oleh `DEFAULT_TICKER` ('BTC-USD'). Apapun yang diketik pengguna akan disimpan dalam variabel `ticker_symbol`. Penting: Meskipun pengguna bisa mengubah nilai ini, model dan scaler yang dimuat di awal (menggunakan `DEFAULT_TICKER`) tidak akan berubah secara otomatis berdasarkan input ini, kecuali logika `load_all_models_and_scalers` diubah untuk menerima `ticker_symbol` sebagai argumen.
 - **`st.sidebar.selectbox(...)`:** Membuat menu dropdown di sidebar dengan label "Periode Grafik Historis". Pengguna dapat memilih salah satu dari opsi dalam list (`["90d", "6mo", "1y", "2y"]`). `index=2` menetapkan '1y' sebagai pilihan default. Pilihan pengguna disimpan dalam variabel `data_period_display`. Nilai ini hanya digunakan untuk menentukan rentang waktu pada grafik historis yang ditampilkan nanti, bukan untuk proses prediksi itu sendiri. `key` adalah identifier unik untuk widget Streamlit.
 - **`st.sidebar.caption(...)`:** Menampilkan teks penjelasan kecil di bawah widget lain di sidebar.

- **Tombol Aksi (st.sidebar.button(...)):** Ini adalah elemen kunci yang memicu logika inti aplikasi. Fungsi `st.sidebar.button()` membuat tombol dengan teks "Muat Data & Prediksi". Fungsi ini mengembalikan `True` hanya pada saat pengguna mengklik tombol tersebut. Oleh karena itu, seluruh blok kode di dalam pernyataan `if` ini hanya akan dieksekusi sebagai respons terhadap klik tombol.

Kondisi Awal (else): Jika tombol belum diklik (saat aplikasi pertama kali dimuat atau setelah rerun tanpa klik tombol), blok `else` akan dieksekusi, menampilkan pesan informatif (`st.info`) kepada pengguna.

Penjelasan Detail (Bagian 7: Logika Utama Aplikasi - Proses Prediksi dan Tampilan Hasil):

Ini adalah blok utama yang dieksekusi saat tombol "Muat Data & Prediksi" ditekan.

- **Pengambilan & Pemrosesan Data:** Memanggil fungsi `get_and_process_data` untuk mendapatkan data terbaru yang sudah diolah dengan fitur-fitur yang relevan.
- **Tampilan Data Kontekstual:** Jika data berhasil didapat, sebagian data terakhir (khususnya fitur input yang benar-benar digunakan, diambil dari `st.session_state`) ditampilkan menggunakan `st.dataframe`. Harga penutupan aktual terakhir juga ditampilkan menggunakan `st.metric` untuk memberi pengguna titik referensi.
- **Persiapan Input Prediksi:** Langkah krusial di mana hanya baris data terakhir (`.iloc[-1:]`) diambil dari `DataFrame` yang diproses. Kolom yang dipilih adalah kolom fitur yang valid dan tersedia (dari `st.session_state.feature_columns_input_actual`). Validasi penting dilakukan untuk memastikan jumlah kolom data ini (harus 16) cocok dengan jumlah fitur yang diharapkan oleh `scaler_X`. Kemudian, `scaler_X.transform()` dipanggil untuk menskalakan baris data tunggal ini menggunakan parameter `scaling` yang telah dipelajari dari data pelatihan.
- **Prediksi Klasifikasi:** Data `X_latest_scaled` (1 baris, 16 fitur, sudah diskalakan) dimasukkan ke metode `.predict()` dari model `model_logreg` dan `model_rf` untuk menghasilkan prediksi kelas (0 atau 1). Metode `.predict_proba()` juga dipanggil untuk mendapatkan probabilitas prediksi untuk masing-masing kelas. Dictionary `direction_map` digunakan untuk menerjemahkan output numerik 0/1 menjadi teks yang lebih mudah dibaca ("Naik" atau "Turun / Sama").
- **Prediksi Regresi & Rekonstruksi Harga:** `X_latest_scaled` dimasukkan ke metode `.predict()` dari model `model_linreg`. Karena model regresi dilatih untuk memprediksi perubahan harga yang diskalakan, outputnya (`scaled_diff_pred`) perlu diubah kembali ke skala dolar asli menggunakan `scaler_y_reg.inverse_transform()`. Hasilnya (`unscaled_diff_pred`) adalah estimasi perubahan harga dalam dolar untuk hari berikutnya. Harga absolut (`predicted_absolute_price`) kemudian diestimasi dengan menjumlahkan `last_actual_close_price` (harga aktual hari ini/terakhir) dengan `unscaled_diff_pred` (prediksi perubahan untuk besok).

- **Tampilan Hasil Prediksi:** Menggunakan kombinasi widget Streamlit untuk menyajikan informasi dengan jelas:
 - `st.subheader` dan `st.markdown` untuk judul dan subjudul bagian.
 - `st.columns(2)` untuk membuat tata letak dua kolom agar hasil LR dan RF bisa dibandingkan berdampingan.
 - `st.metric` untuk menampilkan nilai prediksi utama (arah atau harga estimasi) dengan format yang jelas, termasuk delta untuk menunjukkan perubahan harga estimasi.
 - `st.progress` memberikan representasi visual dari probabilitas prediksi 'Naik'.
 - `st.caption` memberikan detail numerik probabilitas.
 - `st.divider` menambahkan garis pemisah visual antar bagian hasil.
- **Visualisasi Grafik:** Data historis diambil lagi (kali ini durasinya sesuai pilihan `data_period_display` di sidebar). Grafik dibuat menggunakan Matplotlib (`plt.subplots`, `ax.plot`). Grafik ini menampilkan tren harga penutupan historis (`plot_data_to_show['Close']`) dan menambahkan titik merah ('ro') pada tanggal berikutnya (`next_date`) di level `predicted_absolute_price`. Garis bantu horizontal (`axhline`) juga ditambahkan pada level prediksi ini. Setelah grafik selesai dibuat, ia ditampilkan dalam aplikasi Streamlit menggunakan `st.pyplot(fig)`.
- **Penanganan Kesalahan:** Seluruh proses prediksi dan tampilan dibungkus dalam blok `try...except` untuk menangkap error tak terduga dan menampilkannya kepada pengguna (`st.error` dan `st.exception` untuk detail traceback).